

s08 Compact 与 Agent Loop 学习笔记

主题：L1/L2/L3/L4 压缩机制、Python 控制流、agent_loop 正确协议与代码隐患

核心结论：标准答案的主循环结构比你写的版本更接近正确 agent loop；你的版本主要问题是破坏了 tool_use/tool_result 协议、遗漏 stop_reason 结束判断、没有过滤 text block、compact 分支可能重复追加 results。

1. 本次对话覆盖范围

- L1 snipCompact：按 message 数量裁剪中间历史，同时尽量保护 tool_use/tool_result 边界。
- L2 microCompact：不删除 message，只把较早且很长的 tool_result.content 替换为占位符。
- L3 toolResultBudget：最新一轮工具输出太大时，把完整内容落盘，只在上下文保留路径和预览。
- L4 autoCompact/reactiveCompact：用 LLM 做语义总结，把长历史压缩成可继续工作的 summary。
- Python 语法点：切片、sorted(reverse=True)、with/as、json.dumps、except/raise、continue、for...else。
- agent_loop 对比：标准答案与自写版本的差异、为什么某些 continue/else 不能随便改。

思维导图：四层 Compact 总览

上下文压缩 Compact

```
├─ L1 snipCompact：消息级裁剪
│   ├── 保头 + 保尾
│   ├── 中间替换为 [snipped N messages]
│   └── 修补 tool_use/tool_result 边界
├─ L2 microCompact：旧工具结果微压缩
│   ├── 找到所有 user.content 内的 tool_result
│   ├── 保留最近 KEEP_RECENT 个
│   └── 旧且长的 content 替换成占位符
├─ L3 toolResultBudget：最新工具结果预算
│   ├── 只看最后一条 user tool_result message
│   ├── total 超预算时，按内容长度排序
│   └── 大结果写入磁盘，context 里留 path + preview
└─ L4 autoCompact：语义总结
    ├── 先写 transcript 备份
    ├── 调 LLM 总结当前目标/文件/决策/剩余工作
    └── summary 替代原 messages
```

2. L1 snipCompact：裁剪中间 messages

作用：当 messages 数量超过 max_messages 时，保留头部少量消息和尾部近期消息，把中间内容替换成一条占位 user message。

```
def snip_compact(messages, max_messages=50):
    if len(messages) <= max_messages: return messages
```

```
keep_head, keep_tail = 3, max_messages - 3
head_end, tail_start = keep_head, len(messages) - keep_tail
...
return messages[:head_end] + [{"role": "user", "content": f"[snipped {snipped} messages]"}] +
messages[tail_start:]
```

重点：L1 不是总结，而是硬裁剪。它的关键价值不是“压缩得聪明”，而是尽量不把 assistant tool_use 和 user tool_result 拆开。

变量/逻辑	含义	注意点
keep_head	保留开头几条消息	通常保留系统性开端、最初用户目标
keep_tail	保留最近几条消息	最近上下文通常最重要
head_end	头部保留区结束下标	如果最后一条是 tool_use，要继续保留后续 tool_result
tail_start	尾部保留区起始下标	如果第一条是 tool_result，要把前一个 tool_use 也带上
[snipped N messages]	中间被裁掉的占位符	告诉模型中间有历史被省略，但没有语义摘要

隐患：原代码 keep_tail = max_messages - 3，没有给占位 message 留位置，所以最终可能返回 max_messages + 1 条。若要严格不超过上限，应使用 keep_tail = max_messages - keep_head - 1。

3. L2 microCompact: 旧 tool_result 占位

作用：遍历全部 messages，收集 user message 中的 tool_result block；如果工具结果数量超过 KEEP_RECENT，就只保留最近几个原文，较早且较长的结果替换成占位符。

```
def collect_tool_results(messages):
    blocks = []
    for mi, msg in enumerate(messages):
        if msg.get("role") != "user" or not isinstance(msg.get("content"), list): continue
        for bi, block in enumerate(msg["content"]):
            if isinstance(block, dict) and block.get("type") == "tool_result":
                blocks.append((mi, bi, block))
    return blocks

def micro_compact(messages):
    tool_results = collect_tool_results(messages)
    if len(tool_results) <= KEEP_RECENT: return messages
    for _, _, block in tool_results[:-KEEP_RECENT]:
        if len(block.get("content", "")) > 120:
            block["content"] = "[Earlier tool result compacted. Re-run if needed.]"
    return messages
```

表达式	意思
tool_results[:-KEEP_RECENT]	取出除了最近 KEEP_RECENT 个以外的旧工具结果
tool_results[-KEEP_RECENT:]	取最近 KEEP_RECENT 个工具结果
for _, _, block in ...	三元组里 mi、bi 不用，只使用 block
block["content"] = ...	原地修改原始 messages 里的 block 字典

隐患：micro_compact 是原地修改；如果还想保留完整历史，需要 copy.deepcopy(messages)。另外 content 不一定总是字符串，直接 len(block.get("content", "")) 可能对 list/dict 得到错误含义。

4. L3 toolResultBudget: 大工具结果落盘

作用：只处理最后一条 message 中的 tool_result。若它们的 content 总长度超过 max_bytes，就优先把最大的工具输出保存到磁盘文件，并在上下文中仅保留路径和前 2000 字符预览。

```
ranked = sorted(
    blocks,
    key=lambda p: len(str(p[1].get("content", ""))),
    reverse=True
)
```

这句按每个 block 的 content 字符串长度排序，reverse=True 表示从大到小。它不是单纯 reverse 原顺序，而是“按 key 计算出的长度降序排序”。

函数/写法	是否排序	是否改原列表	用途
sorted(x, reverse=True)	是	否，返回新列表	得到一个新的降序列表
x.sort(reverse=True)	是	是	原地降序排序
x.reverse()	否	是	只把原顺序倒过来
reversed(x)	否	否，返回迭代器	倒着遍历

隐患：max_bytes 名字不精确。代码使用 len(str(...))，计算的是字符数近似，不是真正 UTF-8 字节数；中文字符的字节数通常大于字符数。

隐患：tool_use_id 缺失时默认 unknown，多个未知结果可能都写到 unknown.txt；再加上 if not path.exists() 不覆盖，会导致后续内容没有真正保存。

思维导图：L3 处理流程

```
最后一条 messages[-1]
├─ 不是 user 或 content 不是 list? → 原样返回
├─ 找出所有 tool_result block
├─ 计算 total = 所有 content 长度之和
├─ total <= max_bytes? → 原样返回
└─ total > max_bytes
    ├─ 按 content 长度从大到小排序
    ├─ 从最大的 block 开始
    │   └─ content > PERSIST_THRESHOLD?
    │       └─ 否: 跳过
    │       └─ 是: 写入 TOOL_RESULTS_DIR/tool_use_id.txt
    └─ 替换为 <persisted-output> path + preview
```

5. L4 autoCompact / reactiveCompact: LLM 语义总结

autoCompact 与前三层不同。前三层是规则压缩，L4 是语义压缩：把完整历史交给 LLM，总结成能让工作继续的摘要。

```
def write_transcript(messages):
    TRANSCRIPT_DIR.mkdir(parents=True, exist_ok=True)
    path = TRANSCRIPT_DIR / f"transcript_{int(time.time())}.jsonl"
    with path.open("w") as f:
        for msg in messages:
            f.write(json.dumps(msg, default=str) + "\n")
    return path
```

这段代码中，msg 就是 messages 列表中的每一个 message 字典。每个 msg 被 json.dumps 转成一行 JSON 字符串，再通过 f.write 写入 .jsonl 文件。

语法点	解释
with path.open("w") as f	打开文件，并把文件对象临时命名为 f；with 结束后自动关闭文件
as	起别名/绑定变量名；这里把打开后的文件对象绑定到 f
for msg in messages	逐个遍历 messages 里的每一条消息字典
json.dumps(msg, default=str)	把 Python 字典转成 JSON 字符串；遇到不可序列化对象时用 str()
+ "\n"	每条 message 写一行，形成 JSONL 格式

隐患：json.dumps 默认 ensure_ascii=True，中文会显示成 \uXXXX。建议写 transcript 时用 ensure_ascii=False, encoding="utf-8"，便于人类阅读。

隐患：transcript 文件名使用 int(time.time())，秒级精度。一秒内多次压缩可能覆盖文件。建议使用 time.time_ns() 或 uuid。

隐患：summarize_history 只取 json.dumps(messages)[:80000]，保留的是开头，不是最近历史。coding agent 往往最近状态更重要，建议改成“头部 + 尾部”拼接。

6. Python 控制流复盘

6.1 except / raise

except 是捕获异常，raise 是主动抛出异常。except 后再 raise 表示：先记录/处理一下错误，但不吞掉它，继续交给外层处理。

```
try:
    result = risky_function()
except Exception as e:
    print("log:", e)
    raise # 重新抛出刚刚捕获的异常
```

6.2 continue

continue 永远作用于离它最近的循环。

位置	continue 的效果
for block in response.content 内部	跳过当前 block，进入下一个 block
while True 内部、for 外部	结束当前 agent loop 轮次，回到 while 开头重新调用 LLM
if blocked: ... continue	当前工具被 PreToolUse 拦截，返回拦截结果后跳过真实工具执行
for...else 中的 continue	普通工具结果已 append，立即进入下一轮 while

关键判断：看缩进。continue 属于哪一层循环，不看逻辑名字，看它缩进在哪个循环体内部。

6.3 for...else

Python 的 else 不只属于 if，也可以属于 for/while/try。for...else 的规则是：for 正常结束且没有遇到 break，才执行 else；如果被 break 打断，则不执行 else。

```
for block in response.content:
    if block.name == "compact":
        ...
        break
else:
    message.append({"role": "user", "content": results})
    continue
```

这里的 else 用来区分：普通工具路径没有 break，所以统一 append results；compact 特殊路径已经提前 append results 并 break，所以跳过 else，避免重复 append。

思维导图：for...else 在 agent_loop 中的作用

```
for block in response.content
├─ 普通工具调用
│   └─ 执行工具
│       └─ results.append(tool_result)
│           └─ for 正常结束 → 执行 else → append results → 下一轮 while
└─ compact 工具调用
    └─ compact_history
        └─ append compact tool_result
            └─ break → 不执行 else → 避免重复 append
```

7. agent_loop 正确协议

agent_loop 的基本协议可以概括为：LLM 返回 assistant response；如果 stop_reason 是 tool_use，就执行工具；每一个 tool_use 必须回传一个对应 tool_result；然后把 tool_result 作为 user message 追加回 messages，再进入下一轮。

思维导图：标准 agent loop 闭环

```
用户 query / 历史 messages
↓
client.messages.create(messages, tools)
↓
assistant response
├─ stop_reason != tool_use
│   └─ 任务结束，return
└─ stop_reason == tool_use
    └─ append assistant response
        └─ 遍历 response.content
            │   └─ text block: 跳过
            │   └─ tool_use block: 执行对应工具
            └─ append user tool_result message
```

└─ continue while, 重新调用 LLM

点位	标准答案	你原先版本的问题
reactive_retries	函数开头初始化, API 成功后 reset	没有初始化, except 分支会 NameError
stop_reason	非 tool_use 直接 return	注释掉后, 普通文本回答也进入工具循环
block 过滤	if block.type != "tool_use": continue	text block 可能没有 name, 访问 block.name 会报 AttributeError
Unknown tool	也返回 tool_result: Unknown: name	只 print 后 continue, 会导致 tool_use 没有对应 tool_result
compact 分支	用 for...else 避免重复 append	break 后外面仍无条件 append, 可能重复 tool_result
todo reminder	标准答案不含此功能	你插在 API 调用后, 模型本轮看不到 reminder
print output	print(str(output)[:200])	output[:200] 对非字符串不安全

8. 目前修改后代码仍有的隐患与不足

这一章是重点：即使按标准答案修正，你的 s08 agent_loop 仍然有一些工程风险。

隐患	表现	建议
compact 工具协议风险	若 compact_history 先把 messages 替换成 summary, 再追加 compact 的 tool_result, 可能留下没有前置 assistant tool_use 的孤立 tool_result	更安全: 先 append compact tool_result, 再 compact 整个历史; 或 compact 后保留 assistant tool_use + user tool_result 这对消息
L4 摘要可能漏信息	LLM summary 可能漏文件路径、用户约束、最新 bug 状态	保存 transcript 是必要的; summary prompt 应要求列出确定/不确定信息
summarize_history 只取前 80000 字符	可能漏掉最近上下文	改成 raw[:40000] + middle marker + raw[-40000:]
原地修改 messages	L2/L3 都直接改 block["content"], 不可逆	如需完整历史, 先 deepcopy; 或维护 raw transcript
tool_result_budget 只处理最后一条	旧的大工具结果可能仍占上下文	配合 L2; 必要时对历史范围做预算控制
max_bytes 实际是字符数	中文/emoji 字节数与字符数不一致	改名 max_chars, 或用 len(text.encode("utf-8"))
PERSIST 文件名冲突	unknown.txt 或相同 tool_use_id 不覆盖导致内容错配	文件名加入 hash/output digest 或 uuid
transcript 文件名冲突	秒级 timestamp 可能覆盖	使用 time.time_ns() 或 uuid.uuid4().hex
缺少工具异常隔离	handler(**input) 抛异常会中断整个 agent	try/except 包住单个工具, 把错误作为 tool_result 返回
多工具调用 + compact 混合	response 中如果 compact 与其他 tool_use 同时出现, break 会跳过后续工具	规定 compact 必须单独调用, 或检测后只允许 compact 独占
todo reminder 策略粗糙	固定 3 轮提醒可能打扰任务; 且需要放在 API 调用前	用 hook/Stop 或状态机触发, 避免污染工具协议
estimate_size 粗略	字符数估算可能与真实 tokenizer 差距较大	用模型 tokenizer 或保守阈值

9. 推荐的 compact 分支更安全写法

为了避免 compact_history 删除 assistant tool_use 后再追加孤立 tool_result, 可以改成: 先给 compact 工具补上 tool_result, 保证协议完整; 然后再把完整历史总结成一条 summary。

```
if block.name == "compact":
```

```

results.append({
    "type": "tool_result",
    "tool_use_id": block.id,
    "content": "[Compacted. Conversation history has been summarized.]"
})
messages.append({"role": "user", "content": results})

# Now the tool_use/tool_result pair is complete.
# Compact the entire completed history into summary-only context.
messages[:] = compact_history(messages)
break

```

这样下一轮发送给 LLM 的 messages 只有 summary，不会出现 orphan tool_result。缺点是模型看不到原始 compact tool_result，但 summary 中已经包含“历史已压缩”这个事实。

10. 推荐的主循环骨架

```

def agent_loop(messages: list):
    reactive_retries = 0

    while True:
        messages[:] = tool_result_budget(messages)
        messages[:] = snip_compact(messages)
        messages[:] = micro_compact(messages)

        if estimate_size(messages) > CONTEXT_LIMIT:
            messages[:] = compact_history(messages)

        try:
            response = client.messages.create(
                model=MODEL,
                system=SYSTEM,
                messages=messages,
                tools=TOOLS,
                max_tokens=8000,
            )
            reactive_retries = 0
        except Exception as e:
            err = str(e).lower()
            if (("prompt_too_long" in err or "too many tokens" in err)
                and reactive_retries < MAX_REACTIVE_RETRIES):
                messages[:] = reactive_compact(messages)
                reactive_retries += 1
                continue
            raise

    messages.append({"role": "assistant", "content": response.content})

```

```

if response.stop_reason != "tool_use":
    return

results = []
for block in response.content:
    if block.type != "tool_use":
        continue

    if block.name == "compact":
        results.append({"type": "tool_result", "tool_use_id": block.id,
                        "content": "[Compacted.]"})
        messages.append({"role": "user", "content": results})
        messages[:] = compact_history(messages)
        break

    blocked = trigger_hooks("PreToolUse", block)
    if blocked:
        results.append({"type": "tool_result", "tool_use_id": block.id,
                        "content": str(blocked)})
        continue

    handler = TOOL_HANDLERS.get(block.name)
    try:
        output = handler(**block.input) if handler else f"Unknown: {block.name}"
    except Exception as e:
        output = f"Tool error in {block.name}: {e}"

    trigger_hooks("PostToolUse", block, output)
    results.append({"type": "tool_result", "tool_use_id": block.id,
                    "content": str(output)})
else:
    messages.append({"role": "user", "content": results})
    continue

continue

```

11. 最终记忆卡片

概念	一句话记忆
L1 snipCompact	保头保尾，中间剪掉，但不要拆 tool_use/tool_result。
L2 microCompact	旧 tool_result 太长就换占位符，最近的保留。
L3 toolResultBudget	最新工具结果太大就落盘，context 里留路径和预览。
L4 autoCompact	保存 transcript，再让 LLM 总结历史。
reactiveCompact	API 提示上下文过长时，应急总结旧历史，保留最近尾部。
continue	跳到离它最近的循环的下一轮。
for...else	for 没有被 break 打断才执行 else。
tool protocol	每个 assistant tool_use 必须有对应 user tool_result。

stop_reason	不是 tool_use 就 return，说明模型已经给最终回答。
-------------	-----------------------------------

12. 学习路线建议

1. 先把 agent_loop 的“消息协议”画熟：user → assistant tool_use → user tool_result → assistant。
2. 再看控制流：continue/break/for...else 都围绕“是否继续下一轮 LLM 调用”服务。
3. 最后看 compact：L1/L2/L3 是便宜规则压缩，L4 是昂贵但语义更强的 LLM summary。
4. 写代码时每次修改都问：是否留下了孤立 tool_result？是否漏回了某个 tool_use 的结果？是否把普通 text block 当工具处理？